

LAB — Lambda, IAM, SNS e EventBridge

Tarefa 1 — Observar funções do IAM

1.1 — Função IAM do relatório de vendas

Abra o **Console da AWS**.
Vá em **IAM > Funções**.

Na busca, digite:
`sales`

Selecione a função:
salesAnalysisReportRole

Confirme em **Relações de confiança**:

- Entidade confiável: `lambda.amazonaws.com`

Em **Permissões**, confirme as políticas:

- **AmazonSNSFullAccess** → Publicar mensagens no SNS
- **AmazonSSMReadOnlyAccess** → Ler parâmetros do Systems Manager
- **AWSLambdaBasicExecutionRole** → Gravar logs no CloudWatch
- **AWSLambdaRole** → Invocar outra função Lambda

Essa função será usada pela Lambda **salesAnalysisReport**.

1.2 — Função IAM do extrator de dados

Ainda em **IAM > Funções**.

Pesquise novamente:
`sales`

Selecione:
salesAnalysisReportDERole

Confirme em **Relações de confiança**:

- Entidade confiável: `lambda.amazonaws.com`

Em **Permissões**, confirme:

- **AWSLambdaBasicExecutionRole** → Logs no CloudWatch

- **AWSLambdaVPCAccessExecutionRole** → Acesso à VPC

Essa função será usada pela Lambda **salesAnalysisReportDataExtractor**.

Tarefa 2 — Criar camada Lambda e função de extração

2.1 — Criar camada Lambda (PyMySQL)

Abra o **Console da AWS**.

Vá em **Lambda > Camadas**.

Clique em **Criar camada**.

Configure:

- **Nome:** `pymysqlLibrary`
- **Descrição:** PyMySQL library modules
- **Upload:** `pymysql-v3.zip`
- **Runtime compatível:** Python 3.9

Clique em **Criar**.

Confirme:

- Camada criada com sucesso (versão 1)
-

2.2 — Criar função Lambda de extração

Vá em **Lambda > Funções**.

Clique em **Criar função**.

Configure:

- **Criar do zero**
- **Nome:** `salesAnalysisReportDataExtractor`
- **Runtime:** Python 3.9
- **Função de execução:** Usar função existente
- **Função:** `salesAnalysisReportDERole`

Clique em **Criar função**.

2.3 — Adicionar camada à função

Na função `salesAnalysisReportDataExtractor`:

Vá em **Camadas > Adicionar camada**.

Configure:

- Camada personalizada: `pymysqlLibrary`
- Versão: 1

Clique em **Adicionar**.

2.4 — Importar código da função

Na função `salesAnalysisReportDataExtractor`:

Em **Configurações de tempo de execução**:

- **Handler:** `salesAnalysisReportDataExtractor.lambda_handler`

Clique em **Salvar**.

Em **Fonte de código**:

- Carregar de → Arquivo .zip
- Upload: `salesAnalysisReportDataExtractor-v3.zip`

Clique em **Salvar**.

2.5 — Configurar rede (VPC)

Na função `salesAnalysisReportDataExtractor`:

Vá em **Configuração > VPC**.

Configure:

- **VPC:** Cafe VPC
- **Sub-rede:** Cafe Public Subnet 1
- **Grupo de segurança:** CafeSecurityGroup

Clique em **Salvar**.

Tarefa 3 — Testar função de extração

3.1 — Criar evento de teste

Abra **Systems Manager** > **Parameter Store**.

Copie os valores:

- `/cafe/dbUrl`
- `/cafe/dbName`
- `/cafe/dbUser`
- `/cafe/dbPassword`

Volte para **Lambda** > **salesAnalysisReportDataExtractor** > **Teste**.

Crie um novo evento:

- **Nome:** `SARDETestEvent`
- **Modelo:** `hello-world`

Substitua o JSON por:

```
{
  "dbUrl": "valor_dbUrl",
  "dbName": "valor_dbName",
  "dbUser": "valor_dbUser",
  "dbPassword": "valor_dbPassword"
}
```

Clique em **Salvar** e depois em **Testar**.

3.2 — Analisar erro de timeout

Confirme o erro:

- **Task timed out after 3 seconds**

Causa:

- Função não consegue acessar o MySQL na EC2
-

3.3 — Corrigir problema

Abra o **Security Group** da instância EC2 do banco.

Confirme se existe regra de entrada:

- **Porta:** 3306
- **Origem:** Security Group da Lambda

Se não existir:

- Adicione a regra
- Salve

Volte à Lambda e clique em **Testar** novamente.

Confirme:

- Execução bem-sucedida
 - Retorno com status 200
-

3.4 — Criar pedidos no site do café

Abra **EC2 > Instâncias**.

Selecione **CafeInstance**.

Copie o **IPv4 público**.

No navegador, acesse:

`http://IP_PUBLICO/cafe`

Faça alguns pedidos no menu.

Volte para a Lambda e teste novamente.

Confirme:

- JSON retorna produtos e quantidades
-

Tarefa 4 — Criar tópico SNS e inscrição

4.1 — Criar tópico SNS

Abra **SNS > Tópicos**.

Clique em **Criar tópico**.

Configure:

- **Tipo:** Padrão

- **Nome:** salesAnalysisReportTopic
- **Display name:** SARTopic

Crie o tópico e copie o **ARN**.

4.2 — Criar assinatura por e-mail

No tópico:

- Clique em **Criar assinatura**

Configure:

- **Protocolo:** Email
- **Endpoint:** seu e-mail

Confirme a assinatura pelo e-mail recebido.

Tarefa 5 — Criar Lambda principal (salesAnalysisReport)

5.1 — Conectar na instância CLI

Abra **EC2 > Instâncias**.

Selecione a instância **CLI Host**.

Clique em **Conectar > EC2 Instance Connect**.

5.2 — Configurar AWS CLI

No terminal:

```
aws configure
```

Informe:

- Access Key
 - Secret Key
 - Região: us-west-2
 - Output: json
-

5.3 — Criar função Lambda via CLI

Confirme que o arquivo existe:

```
cd activity-files  
ls
```

Execute:

```
aws lambda create-function \  
--function-name salesAnalysisReport \  
--runtime python3.9 \  
--zip-file fileb://salesAnalysisReport-v2.zip \  
--handler salesAnalysisReport.lambda_handler \  
--region us-west-2 \  
--role ARN_DA_FUNCAO_IAM
```

5.4 — Configurar variável de ambiente

Na função `salesAnalysisReport`:

Vá em **Configuração > Variáveis de ambiente**.

Adicione:

- **Key:** `topicARN`
- **Value:** ARN do tópico SNS

Salve.

5.5 — Testar função principal

Crie evento de teste:

- **Nome:** `SARTestEvent`
- **Modelo:** `hello-world`

Clique em **Testar**.

Confirme:

- Execução bem-sucedida
 - E-mail recebido com relatório
-

Tarefa 6 — Agendar execução com EventBridge

Na função **salesAnalysisReport**:

Clique em **Adicionar gatilho**.

Configure:

- **Origem:** EventBridge
- **Nova regra**
- **Nome:** salesAnalysisReportDailyTrigger
- **Tipo:** Expressão de agendamento

Exemplo (teste em 5 minutos – UTC):

```
cron(35 16 ? * MON-SAT *)
```

Adicione o gatilho.

Confirme:

- E-mail recebido automaticamente no horário agendado